# MARLET: Multi-Agent Retrieval for Learning-State-Aware Educational Tutoring

Anonymous Authors

Abstract— Large language models can generate fluent tutoring responses, but they need grounding that respects both instructional evidence and the learner's current state. Classical retrieval-augmented generation (RAG) grounds generation through a fixed retrieve-then-read controller: the query is matched against a corpus, and the generator reads the returned passages. That design is often insufficient for tutoring, where the same surface question can require different evidence, scaffolding, or rubric constraints for different learners. We propose MARLET (Multi-Agent Retrieval for Learning-State-Aware Educational Tutoring), a framework that places a supervisory router, planner, diagnoser, rubric module, retriever, tutor, and critic around a fixed downstream tutor. The framework first builds a tutoring brief from the sample, inferred learning state, and answer criteria; retrieval is then invoked only when the route indicates that external evidence is needed. We validate the framework against classical RAG under matched backbones, corpus, reranker, and response budget on TutorEval and EduBench. MARLET improves fixed-tutor quality metrics over classical RAG while using fewer tokens; on EduBench it also reduces latency and backbone-call count. These results support the framework claim: for educational tutoring, grounding should be controlled by pedagogical state and task regime, not by the surface query alone.

## I. Introduction

Large language models (LLMs) are increasingly used as educational tutors because they can generate explanations, answer questions, diagnose errors, and provide feedback in natural language. However, an LLM tutor cannot rely only on its parametric knowledge. Educational responses often need to be grounded in specific instructional materials, such as textbook chapters, course notes, rubrics, worked examples, or learner profiles. Without such grounding, the tutor may produce fluent responses that are misaligned with the curriculum, learning objectives, or student context.

Classical retrieval-augmented generation (RAG) has therefore become the default mechanism for grounding LLM-based tutoring [1]. In classical RAG, the tutor retrieves evidence primarily from the query representation: the input query is embedded, the top-$k$ most similar educational passages are retrieved, and the LLM generates a response conditioned on those passages. Thus, the quality and relevance of the retrieved evidence largely depend on how the tutoring need is represented in the query.

Educational tutoring is more complex than factual question answering. A student's question is often only a partial signal of what the tutor needs to know. The appropriate response may depend on the learner's current level, goals, weak points, recent confusion, prior dialogue, and likely misconceptions. We refer to this student-specific information as the learner's personalized learning state. Two learners may ask the same question but require different tutoring actions. Therefore, the retrieval problem in tutoring is not simply to find passages that match the topic of the question, but to find evidence that is pedagogically appropriate for the learner.

One possible workaround for this challenge is to append learner state, dialogue history, grading criteria, and task constraints directly to the retrieval query. However, this shifts the burden onto a single query embedding. As more conditions are added, the embedding must compress heterogeneous information: the topic, the learner's level, the suspected misconception, the instructional goal, and the response constraints. This can blur the search intent, complicate query construction, and produce retrieval results that satisfy some needs while missing others. In this sense, personalization cannot be solved reliably by simply making the retrieval query longer.

Therefore, we propose MARLET (Multi-Agent Retrieval for Learning-State-Aware Educational Tutoring) as an augmented RAG framework for this setting. Instead of letting the retriever act as the top-level controller, a supervisory router first identifies the tutoring regime and decides whether retrieval is needed. A planner scopes the search intent, a diagnoser summarizes visible learning state, and a rubric module states what the answer must satisfy. These modules run before the fixed tutor writes, so the retriever becomes a conditional tool inside an inspectable tutoring controller rather than a mandatory first step.

The experiments are designed to validate the framework, not to present a broad model leaderboard. We therefore compare the proposed framework with a classical retrieve-then-read RAG baseline while holding the backbone, corpus, reranker, response cap, tutor, and critic fixed. The baseline is not meant to exhaust every possible prompt-augmented RAG variant; it tests the standard design in which retrieval is driven before an explicit learner-state, plan, and rubric brief is constructed. Our validation on TutorEval and EduBench shows that the same tutor scores higher when fed context assembled by MARLET; on EduBench it also reduces latency and backbone calls.

Submitted to the 2026 IEEE International Conference on Systems, Man, and Cybernetics (SMC). Single-blind review.

Contributions. (i) We propose MARLET, a learning-state-aware multi-agent retrieval framework for educational tutoring, with an explicit supervisory router, planner, diagnoser, rubric module, conditional retriever, tutor, and critic. (ii) We formalize the router and retrieval gate as transparent, training-free controls that determine when learner state, rubric constraints, and external evidence should enter the tutoring brief. (iii) We validate the framework against classical RAG on two educational tutoring benchmarks under shared backbone, corpus, reranker, and response budget, reporting both tutoring quality and resource trade-offs.

## II. Related Work

Classical RAG fixes retrieval as the first controller: the query is matched to passages, and generation conditions on the returned evidence [1]. Selective and corrective RAG methods improve this controller. Self-RAG learns when to retrieve, generate, and critique; CRAG evaluates retrieved documents and attempts correction when retrieval is unreliable; Adaptive-RAG and TARG vary retrieval behavior by query complexity or retrieval utility [2]–[5]. These methods address an evidence-control problem: whether the passages retrieved for a query are necessary, sufficient, or reliable. They still treat the query as the main signal for retrieval control. They do not infer the learner's stage before retrieval, separate rubric constraints from topic evidence, or decide whether external passages are the right source of grounding for a tutoring decision.

Agentic RAG and multi-agent LLM systems move beyond a single retrieve-then-read step. ReAct and AutoGen show that tool use and role decomposition can improve complex task execution [6], [7]. MA-RAG uses multiple agents for query disambiguation, evidence extraction, and answer synthesis on ambiguous or multi-hop QA tasks [8]. These systems demonstrate coordination benefits, but their roles are designed for general task execution or ambiguous question answering. They do not define pedagogical regimes, infer visible learner state, or bind the final response to rubric-style answer criteria. As a result, they can coordinate tools without making the educational reason for retrieving, skipping retrieval, or prioritizing rubric context inspectable.

Recent educational work shows that tutoring quality requires more than answer accuracy. AgentTutor builds a multi-turn personalized learning system with learner assessment, strategy selection, reflection, and memory [9]; adaptivity studies show that LLM tutors are sensitive to omitted student-context features and still struggle to match the adaptivity of intelligent tutoring systems [10]; tutoring benchmarks emphasize pedagogy, mistake diagnosis, guidance quality, and learner support rather than short-form answer accuracy alone [11]–[14]. These works define important tutoring requirements, but they do not formulate retrieval as learner-state-aware control. Existing systems evaluate pedagogy or maintain interaction state, yet they do not expose retrieval as a transparent decision about when to retrieve, what evidence to retrieve, and how much context to allocate to evidence rather than learner-state and rubric information.

## III. Methodology

The proposed framework, MARLET, is a tutoring pipeline organized around a supervisory controller. For each sample, the controller reads the question, dialogue, context text, rubric items, and metadata, then assigns a pedagogical regime and decides which modules should run. The planner states the instructional goal and search intent, the diagnoser summarizes visible learning state, the rubric module lists answer criteria, and retrieval is invoked only when external evidence is needed. Fig. 1 sketches this pipeline.

The module boundaries come from observed benchmark patterns. TutorEval samples such as "Which is the fastest mode of heat transfer?" require the answer radiation, the explanation that no medium is required, and alignment with chapter evidence; a thermal-conductivity-unit sample requires isolating $k$ and reasoning from SI units. These cases motivate a planner that narrows evidence search, a retriever that supplies chapter grounding, and a rubric module that preserves expected teaching points. EduBench supplies different signals: a personalized-learning sample describes a grade-five Chinese learner who likes folklore but struggles with pronunciation, while an emotional-support sample describes anxiety about speaking in class. These motivate a diagnoser that captures visible learning state and affect before generation. EduBench grading and error-correction samples require a score, correction, and personalized feedback, motivating rubric constraints and the critic. Thus MARLET routes by grounding need rather than treating every sample as query-only retrieval.

The design principle is simple: retrieve when external evidence is needed, and otherwise spend context on learner state and answer criteria. Validation keeps the tutor/critic backbone, corpus, reranker, packer, and response budget fixed so that the classical RAG comparison tests how context is assembled before the same tutor writes.

### A. Supervisory Routing

The supervisor is the framework's decision-making component. It estimates whether a sample primarily needs external evidence, pedagogical coordination, rubric feedback, planning, or dialogue-sensitive tutoring. The reported runs use transparent heuristics rather than a trained router. For a standardized benchmark sample $x$, the router first builds a normalized text blob $b(x)$ from the question, optional context, rubric text, and dialogue.

The first routing quantity is a cue-hit rate. For each cue phrase $u$, define $m(u, x) = 1$ if $u$ appears in $b(x)$, and

TABLE I

Framework components and responsibilities.

| Component | Responsibility |
|---|---|
| Supervisor | Assigns pedagogical regime and activates retrieval, rubric, and critic switches. |
| Planner | Converts the tutoring request into an operating plan and up to three scoped retrieval queries. |
| Diagnoser | Infers visible learning state from the prompt and dialogue, including level and likely misconceptions. |
| Rubric module | Summarizes explicit rubric items or default answer-quality criteria. |
| Retriever | Runs only when requested by the route or fallback gate, then reranks and packs evidence. |
| Tutor/Critic | Generates the final response and checks evidence markers, rubric coverage, and pedagogy. |

$m(u, x) = 0$ otherwise. For a cue family $G$, the router computes

$$\kappa_G(x) = \frac{\sum_{u \in G} m(u, x)}{|G|}, \tag{1}$$

where $|G|$ is the number of cue phrases in that family. Thus $\kappa_G(x)$ is just the fraction of cues that were observed: 0 means no cue matched, and 1 means every cue matched. The cue families are evidence ($E$), coordination ($C$), rubric ($R$), planning ($P$), and tutoring ($T$).

The cue-hit rate is then converted into a routing score:

$$s_j(x) = \min\{1, \max\{0, \kappa_j(x) + B_j(x) + P_j(d_x)\}\}, \tag{2}$$

where $j \in \{e, c, r, p, t\}$ indexes evidence, coordination, rubric, planning, and tutoring, and $\kappa_j(x)$ means the cue-hit rate for that cue family. The term $B_j(x)$ is a field bonus from visible sample structure: context or image fields increase the evidence score, rubric fields increase the rubric score, and longer dialogue increases the tutoring score. The term $P_j(d_x)$ is a dataset prior from the dataset label $d_x$; for example, evidence-grounded datasets receive an evidence prior. The outer min / max only keeps the score between 0 and 1. These scores are not probabilities; they are transparent control signals used to decide which modules should run.

The router then assigns a pedagogical regime: lesson planning, rubric feedback, adaptive tutoring, or evidence-grounded reasoning. If the adapter supplies a regime hint, the supervisor uses it; otherwise it selects lesson planning when $s_p$ is strongest and at least 0.42, rubric feedback when $s_r \geq \max\{s_t, 0.35\}$, adaptive tutoring when $s_t \geq 0.35$, and evidence-grounded reasoning otherwise. Retrieval is controlled by the binary gate

$$g(x) = \begin{cases} 1, & s_e(x) \geq 0.35 \text{ or } \text{regime}(x) = \text{EG}, \\ 0, & \text{otherwise.} \end{cases} \tag{3}$$

Here $s_e(x)$ is the evidence score and $\text{regime}(x) = \text{EG}$ means evidence-grounded reasoning. In words, retrieval is requested when the sample looks evidence-heavy or the regime itself is evidence-grounded. If $g(x) = 0$, the framework still builds the tutoring brief but skips retrieval. If no chunks have been retrieved and the evidence score is at least 0.45, one fallback retrieval pass is issued using the raw question. Thus the router constrains both how much pedagogical coordination to activate and whether the retriever should be used.

## B. Specialist Agents and Shared Context

All implemented mechanisms operate over the same tutoring prompt, inferred learning state, rubric summary, retrieved evidence, and draft answer. The tutor and the critic are held constant across all implemented mechanisms: the tutor writes from the supplied context, and the route-controlled critic may revise the draft when the regime calls for adaptive tutoring. The only moving parts are the upstream context builders. In the reported configuration, planner, diagnoser, and rubric are deterministic heuristics. Their roles are asymmetric. The planner alone is retrieval-facing: it emits up to three deduplicated queries from the raw question, extracted key terms, dataset cueing, and the first rubric items. The diagnoser and rubric module are tutor-facing: they summarize transient learning state and answer criteria for the tutor prompt, but they do not alter retriever scoring. In this release, learning state is inferred from the current question and dialogue rather than from a persistent profile store.

The tutor itself is the LLM-driven agent that produces the final response. It is dataset-aware but architecturally fixed: its prompt includes the dataset name, question, recent dialogue, inferred learning state, operating plan, rubric summary, and either retrieved evidence or inline benchmark context. It is instructed to be correct, pedagogically useful, concise, and to end with a next step or check-for-understanding when appropriate. Retrieved evidence must be cited with [doc_id] markers. EduBench consensus rows require a JSON object with a score, scoring details, and personalized feedback; TutorEval and the other tutoring-style profiles use instructional prose. This design isolates the effect of changing the retrieval context while leaving the downstream writer unchanged; personalization reaches retrieval mainly through planner query formulation, not through profile-aware reranking.

## C. Hybrid Retrieval, Reranking, and Context Packing

The retrieval stack is intentionally lightweight and also follows the codebase exactly. The following equations are scoring rules, not learned models: a larger score means the chunk is ranked higher for the current query. For a query $q$ and candidate chunk $c$, the hybrid index first computes

$$\begin{aligned} \text{score}_0 = 0.58\, k_w + 0.17\, k_{ch} \\ + 0.25\, k_\ell + \beta, \end{aligned} \tag{4}$$

where all terms are computed for the current $(q, c)$ pair. Here $k_w$ is word-level TF–IDF similarity, $k_{ch}$ is character 3–5 gram TF–IDF similarity, and $k_\ell$ is the similarity in the truncated-SVD latent space. The weights state

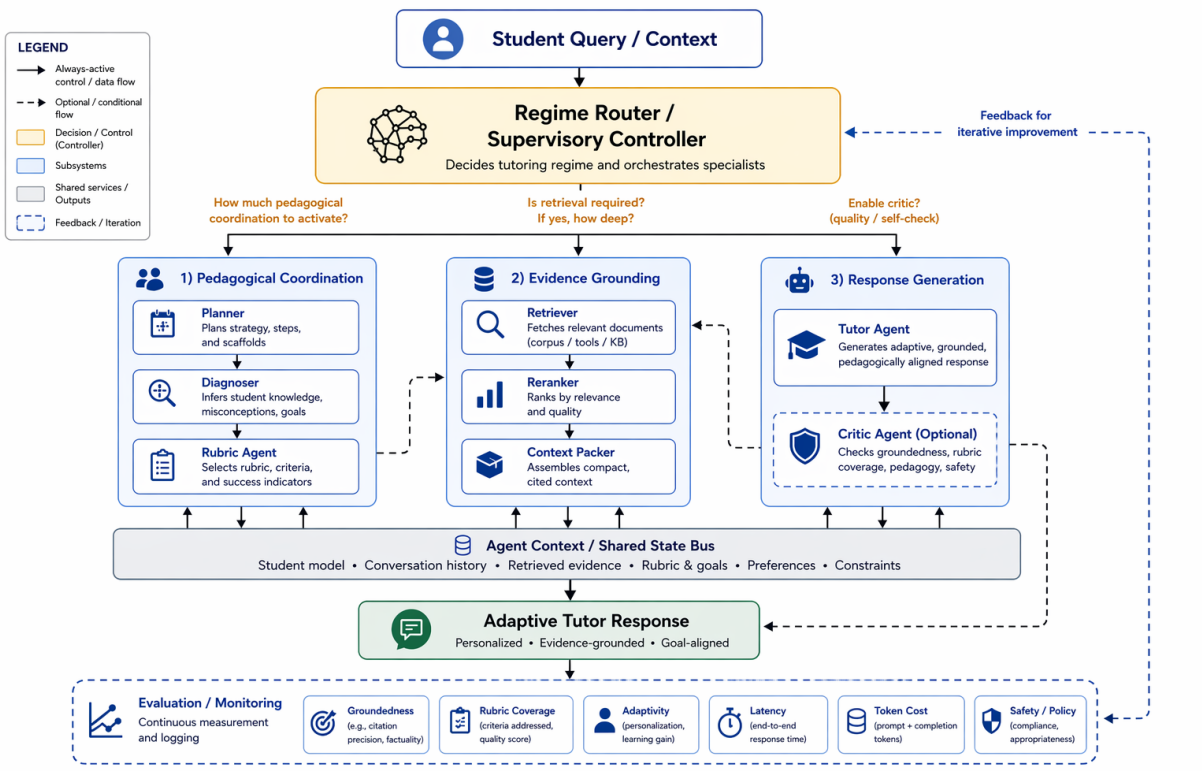


Fig. 1. Main framework of MARLET. The supervisory router scores the sample, assigns the pedagogical regime, and decides whether retrieval is needed before planner, diagnoser, and rubric modules assemble the tutoring brief.

how much the released retriever trusts each signal: word similarity contributes most, latent semantic similarity is second, and character similarity helps with spelling variants and short phrases. The query boost $\beta$ adds 0.01 per overlapping title token and, in citation-seeking mode, an additional 0.025 for chunks whose source type is reference, lecture, or document. When multiple planner queries are issued, the retriever merges all returned candidates and deduplicates them by chunk identifier before reranking.

In the released paper configuration, no trained reranker is loaded, so the heuristic reranker score is

$$\begin{aligned}\text{score}_1 = 0.55\,\text{score}_0 + 0.20\,o_t + 0.10\,o_h \\ + 0.06\,m_p + 0.05\,o_n + 0.04\,\rho,\end{aligned} \tag{5}$$

where all terms are again computed for the current $(q, c)$ pair. The variables $o_t$, $o_h$, $m_p$, and $o_n$ denote token overlap, title overlap, exact phrase-match score, and numeric overlap, and $\rho$ is a brevity prior. This reranker favors chunks that match the question text, title, exact phrases, and numbers while mildly preferring concise chunks. Once planner queries are merged, reranking remains question-centric and does not consume the learner summary directly.

Final context packing balances relevance against redundancy:

$$c^\star = \arg\max_{c \in \mathcal{C} \setminus S} \lambda\,\text{score}_1(q, c) - (1-\lambda) \max_{c' \in S} \text{overlap}(c, c'), \tag{6}$$

where $c^\star$ is the next selected chunk, $\mathcal{C}$ is the reranked candidate pool, $S$ is the set of already packed chunks, and $\text{overlap}(c, c')$ is the redundancy term between two chunks. The first term rewards relevance to the query; the second term penalizes selecting another chunk that repeats content already in $S$. With $\lambda = 0.68$, a four-chunk cap, and a 6000-character context budget, this keeps the comparison focused on retrieval quality rather than on simply enlarging the context window.

## D. Evaluation Protocol

Evaluation is dataset-aware, but the reported metrics are internal automatic proxies implemented in the released evaluator rather than official benchmark scoring scripts. Token-F1 is the token-overlap F1 between the model answer and the benchmark reference answer. Rubric coverage is the fraction of rubric items the answer covers, using either near-verbatim phrase match or sufficiently strong token overlap anchored by a content-bearing token. Latency, total tokens, and backbone-call count capture inference cost; backbone calls are the logged number of LLM requests needed for one sample.

For EduBench, we additionally report EB12D, the mean over 12 rubric-oriented signals spanning scenario adaptation, factual reasoning, and pedagogical application. Those signals reward output-format compliance, supportive but non-negative tone, relevance to the question, use of scenario elements from the sample, alignment with the benchmark score, domain-knowledge grounding, reasoning quality, error identification, clarity,

motivational guidance, personalization, and higher-order prompting. We report EduAlign as $1 - |s(y) - \bar{s}|/100$, where $y$ is the model's JSON output, $s(y)$ is the numeric score extracted from it, and $\bar{s}$ is the benchmark reference score.

For TutorEval, we report tutor-hit as the mean key-point credit over the $K$ expected teaching points, with full credit for explicit coverage, partial credit for moderate token overlap, and zero otherwise. We also track a chapter-grounding proxy by measuring whether the response stays aligned with retrieved evidence when retrieval is used, or with the inline chapter text otherwise. Concretely, EduBench is evaluated as a structured educational-judgment task, whereas TutorEval is evaluated as a chapter-grounded tutoring-response task. In the per-example evaluator outputs, off-profile metrics remain present as zeros; in the paper tables, benchmark-inapplicable columns are simply not shown. The released code also records a workload audit $U = T + 160Q_r + 90C_r + 240N_a + 30E_v$, where $T$ is total tokens, $Q_r$ retrieval queries, $C_r$ retrieved chunks, $N_a$ materialized agent outputs, and $E_v$ trace events. This score is used only as an efficiency audit, not as a training objective. A retrieval-agnostic corpus-factuality check is supported when a compatible shared index is available, but the archived main comparison predates that audit, so we do not present it as a headline result here.

## IV. Validation Experiments

We validate the framework on two public educational benchmarks. TutorEval [11] contains 834 expert-written science tutoring questions paired with long STEM chapters and teaching key points, so responses must explain, disambiguate, and stay chapter-grounded. EduBench [12] covers nine educational scenarios, including question answering, error correction, personalized support, emotional support, grading, teaching-material generation, and personalized content creation. Its rows contain prompts, scenario metadata, and reference model predictions; our adapter converts those references into consensus-style supervision through score statistics and rubric terms. For reporting, we use the 1562-example four-way EduBench intersection completed by all forced architectures.

The primary validation compares classical RAG with MARLET under matched Qwen3.5-4B infrastructure, reasoning budget 512, temperature 0.15, response cap 900, shared corpus, shared reranker, and fixed retrieval hyperparameters. Both systems run on the same benchmark samples. In classical RAG, retrieval is driven by the raw question before tutoring; the tutor then receives ordinary sample fields and retrieved chunks, but not the planner, diagnoser, or rubric brief. This is the intended contrast: the baseline tests retrieve-then-read grounding, while the proposed framework tests explicit context construction plus conditional retrieval.

TABLE II

Quality and 4090 resource summary. Quality is higher-better; runtime, tokens, and calls are lower-better.

| Data | Metric | RAG | Ours | Δ |
|---|---|---|---|---|
| TutorEval | Token-F1 | 0.112 | 0.115 | $+0.004^{\star}$ |
| | TE hit | 0.938 | 0.957 | $+0.019^{\star\star}$ |
| | Rubric | 0.919 | 0.944 | $+0.026^{\star\star}$ |
| | Latency (s) | 51.90 | 49.11 | $-2.79$ |
| | Tokens | 3975 | 3490 | $-484^{\star\star\star}$ |
| | Calls | 1.84 | 1.56 | $-0.28$ |
| EduBench | EB12D | 0.367 | 0.398 | $+0.031^{\star\star\star}$ |
| | Rubric | 0.705 | 0.891 | $+0.186^{\star\star\star}$ |
| | EduAlign | 0.166 | 0.126 | $-0.040^{\star\star}$ |
| | Latency (s) | 19.83 | 13.90 | $-5.92^{\star\star\star}$ |
| | Tokens | 4020 | 2226 | $-1794^{\star\star\star}$ |
| | Calls | 1.97 | 1.48 | $-0.49^{\star\star\star}$ |

$^{\star\star\star}p<10^{-4}$, $^{\star\star}p<0.01$, $^{\star}p<0.05$. Runtime is mean wall-clock seconds per sample from paired 4B runs on the 4090 server.

The validation is fair at the infrastructure level but intentionally not identical at context construction: giving classical RAG the planner, diagnoser, and rubric outputs would convert it into the proposed framework, while withholding them from MARLET would remove the claimed contribution. We therefore report quality and resource outcomes together.

### A. Framework Validation

Table II reports paired matched-sample comparisons with bootstrap confidence intervals and permutation p-values. On TutorEval, MARLET raises Token-F1, tutor-hit, and rubric coverage while using fewer tokens and fewer backbone calls; latency trends lower but is not reliable ($p$=0.21). Because the forced TutorEval harness computes route metadata before architecture forcing, the proposed framework retrieves on every TutorEval sample; this result tests whether planning, diagnosis, and rubric context help even when chapter evidence is always consulted.

### B. Trade-offs and Ablations

For TutorEval, the 95% confidence intervals are $[0.0002, 0.007]$ for Token-F1, $[0.006, 0.032]$ for tutor-hit, and $[0.006, 0.044]$ for rubric coverage. For EduBench, they are $[0.027, 0.036]$ for EB12D, $[0.174, 0.198]$ for rubric coverage, and $[-0.069, -0.011]$ for EduAlign; cost intervals are $[-6702, -5248]$ ms for latency, $[-1847, -1741]$ tokens, and $[-0.52, -0.46]$ calls. Classical RAG keeps a small EduAlign edge, but the frozen hybrid skips retrieval on every EduBench sample, so these prompts are usually better served by learner- and rubric-centered coordination than by external lookup.

The framework can use more modules than classical RAG, so resource cost must be reported. In these runs, scoped planner queries and the retrieval gate reduce token use; retrieval-free diagnostics remain unsuitable as main baselines for TutorEval because chapter evidence is required. Two 4B ablations on a shared TutorEval

slice ($n$=100) further isolate the mechanism: forcing retrieval hurts Token-F1 ($-0.003$), tutor-hit ($-0.015$), and rubric coverage ($-0.025$), while disabling the shared critic shows that the main gain does not come from critic revision. On Qwen3.5-27B-FP8, TutorEval deltas collapse near zero but hybrid remains more efficient; EduBench 27B repeats the pattern with EB12D $+0.032$, rubric $+0.210$, tokens $-1{,}861$, and latency $-22{,}070$ ms.

These results clarify the intended use of MARLET: it is not a universal replacement for retrieval, but a controller for deciding what grounding a tutoring sample needs. Chapter-grounded samples still need evidence, whereas learner-profile, rubric, or scenario-heavy samples may be better served by learner-state and answer-criteria summaries than by mandatory topical retrieval.

## V. Limitations

The study validates a framework rather than a final tutoring product. It reports automatic rather than human evaluation and is limited to two open-weight backbones, English-only data, benchmark outputs rather than downstream learning gains, and visible learner-state inference from prompts and dialogue. Real deployment would require privacy safeguards, bias checks, data governance, and human instructional oversight.

## VI. Conclusion

MARLET reframes grounding for educational tutoring as supervised context construction rather than query-only passage lookup. Under shared infrastructure, it improves fixed-tutor quality and efficiency over classical RAG, showing that learner state, rubric constraints, and conditional evidence should be coordinated before the tutor writes. Future work should compare against stronger controller-based RAG baselines, add human and multi-turn evaluation, and extend the router with learned calibration, profile-aware reranking, multilingual curricula, and privacy-preserving learner-state inference.

## References


[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[2] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in International Conference on Learning Representations, 2024. [Online]. Available: https://arxiv.org/abs/2310.11511

[3] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective retrieval augmented generation," arXiv:2401.15884, 2024. [Online]. Available: https://arxiv.org/abs/2401.15884

[4] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. Park, "Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity," in Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers). Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 7036–7050. [Online]. Available: https://aclanthology.org/2024.naacl-long.389/

[5] Y. Wang, L. Wei, and H. Ling, "Retrieval as a decision: Training-free adaptive gating for efficient RAG," arXiv:2511.09803, 2026. [Online]. Available: https://arxiv.org/abs/2511.09803

[6] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in International Conference on Learning Representations, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[7] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2024. [Online]. Available: https://arxiv.org/abs/2308.08155

[8] T. Nguyen, P. Chin, and Y.-W. Tai, "MA-RAG: Multi-agent retrieval-augmented generation via collaborative chain-of-thought reasoning," arXiv:2505.20096, 2025. [Online]. Available: https://arxiv.org/abs/2505.20096

[9] Y. Liu, Z. Song, J. Lou, C. Wu, and J. Li, "AgentTutor: Empowering personalized learning with multi-turn interactive teaching in intelligent education systems," arXiv:2601.04219, 2026. [Online]. Available: https://arxiv.org/abs/2601.04219

[10] C. Borchers and T. Shou, "Can large language models match tutoring system adaptivity? A benchmarking study," in Artificial Intelligence in Education. Springer Nature Switzerland, 2025, pp. 407–420. [Online]. Available: https://arxiv.org/abs/2504.05570

[11] A. Chevalier, J. Geng, A. Wettig, H. Chen, S. Mizera, T. Annala, M. J. Aragon, A. R. Fanlo, S. Frieder, S. Machado, A. Prabhakar, E. Thieu, J. T. Wang, Z. Wang, X. Wu, M. Xia, W. Xia, J. Yu, J.-J. Zhu, Z. J. Ren, S. Arora, and D. Chen, "Language models as science tutors," arXiv:2402.11111, 2024. [Online]. Available: https://arxiv.org/abs/2402.11111

[12] B. Xu, Y. Bai, H. Sun, Y. Lin, S. Liu, X. Liang, Y. Li, Z. Dong, J. Zhang, Y. Deng, X. Zou, Y. Gao, and H. Huang, "EduBench: A comprehensive benchmarking dataset for evaluating large language models in diverse educational scenarios," arXiv:2505.16160, 2025. [Online]. Available: https://arxiv.org/abs/2505.16160

[13] J. Macina, N. Daheim, I. Hakimi, M. Kapur, I. Gurevych, and M. Sachan, "MathTutorBench: A benchmark for measuring open-ended pedagogical capabilities of LLM tutors," in Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 204–221. [Online]. Available: https://aclanthology.org/2025.emnlp-main.11/

[14] R. S. Srinivasa, Z. Che, C. B. C. Zhang, D. Mares, E. Hernandez, J. Park, D. Lee, G. Mangialardi, C. Ng, E.-Y. H. Cardona, A. Gunjal, Y. He, B. Liu, and C. Xing, "TutorBench: A benchmark to assess tutoring capabilities of large language models," arXiv:2510.02663, 2025. [Online]. Available: https://arxiv.org/abs/2510.02663